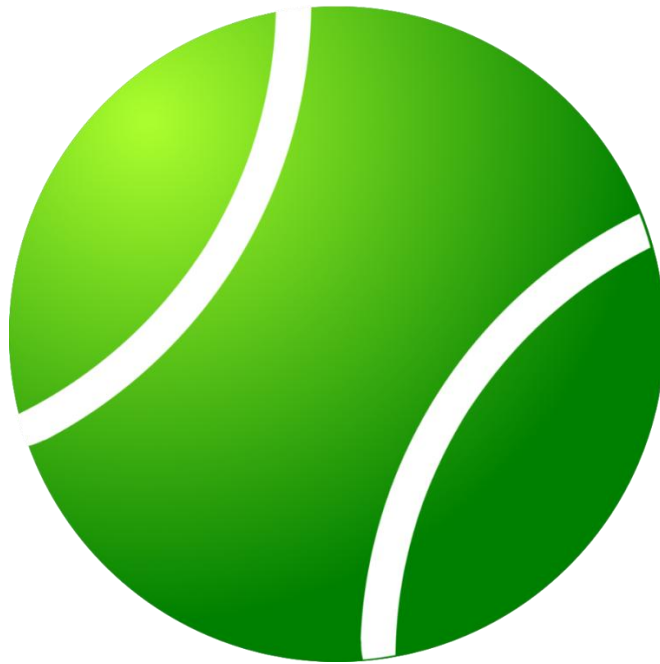


Tennisklubben Smash

Gruppe 7

Ousman (ouba1000) | Nicklas C. (ClausenX)

Morten (mtothejensen) | Amanda (AmandaHult98)



GitHub Repo

<https://github.com/ClauseNEK/Tennisklubben>

Kravsspecifikation (funktionel/ikke-funktionelle)

Funktionelle krav (FK)

FK-01: Systemet skal kunne registrere og redigere et medlems stamoplysninger (medlems-ID, navn, alder) samt slette medlemmer ud fra medlems-ID.

FK-02: Systemet skal kunne vise medlemslisten sorteret alfabetisk efter navn og efter alder (stigende)

FK-03: Systemet skal kunne registrere et medlems aktivitetsform (aktiv/passiv), aldersgruppe (junior under 18 / senior 18+) og spillertype (motionist/konkurrence), og denne kategorisering skal kunne ændres efter oprettelse.

FK-04: Systemet skal automatisk beregne årligt kontingent efter satserne: aktiv junior 800 kr., aktiv senior (18-59) 1500 kr., aktiv senior 60+ 1500 kr. med 25 % rabat (1125 kr.), passiv 250 kr.

FK-05: Systemet skal kunne vise en samlet liste over forventede indbetalinger sorteret efter beløb.

FK-06: Systemet skal kunne vise en separat restanceliste sorteret efter beløb og navn, og kassereren skal kunne markere et medlem som "Betalt", hvorefter payments.csv opdateres.

FK-07: Systemet skal give formanden en advarsel når et medlem nærmer sig en aldersgrænse, der ændrer kontingentkategori (17 år før junior→senior og 60 år før seniorrabat).

FK-08: Systemet skal kunne registrere et træningsresultat for en konkurrencespiller med disciplin (single, double, mixed double), bedste resultat og dato.

FK-09: Systemet skal kunne vise et top 5-leaderboard per disciplin, opdelt på junior og senior, baseret på bedste registrerede træningsresultat, og listen skal opdateres automatisk ved nye resultater.

FK-10: Systemet skal automatisk indlæse data fra members.csv, payments.csv, training_results.csv og tournaments.csv ved opstart og gemme ændringer tilbage, så data ikke går tabt mellem sessioner.

Ikke-funktionelle krav (IFK)

IFK-01: Systemet skal designes og kodes efter objektorienterede principper, så koden er genbrugelig, vedligeholdelsesvenlig og robust.

IFK-02: Designet skal følge både GRASP- og SOLID-principperne (jf. afsnittet "GRASP & SOLID").

IFK-03: Relaterede klasser skal opdeles logisk i packages (fx model, service, ui, filehandler), så ansvar er adskilt mellem lagene.

IFK-04: Systemet skal anvende selvdefinerede Java-interfaces (fx Membership, AgeValidator, Logger), og klasser skal afhænge af interfaces frem for konkrete implementeringer.

IFK-05: Systemet skal have en brugergrænseflade med menustruktur og underfunktioner pr. brugerrolle (formand, kasserer, coach), der dækker al funktionalitet fra opgavebeskrivelsen.

IFK-06: Brugergrænsefladen skal anvende farvekodning til at fremhæve menuvalg, advarsler og fejlbeskeder.

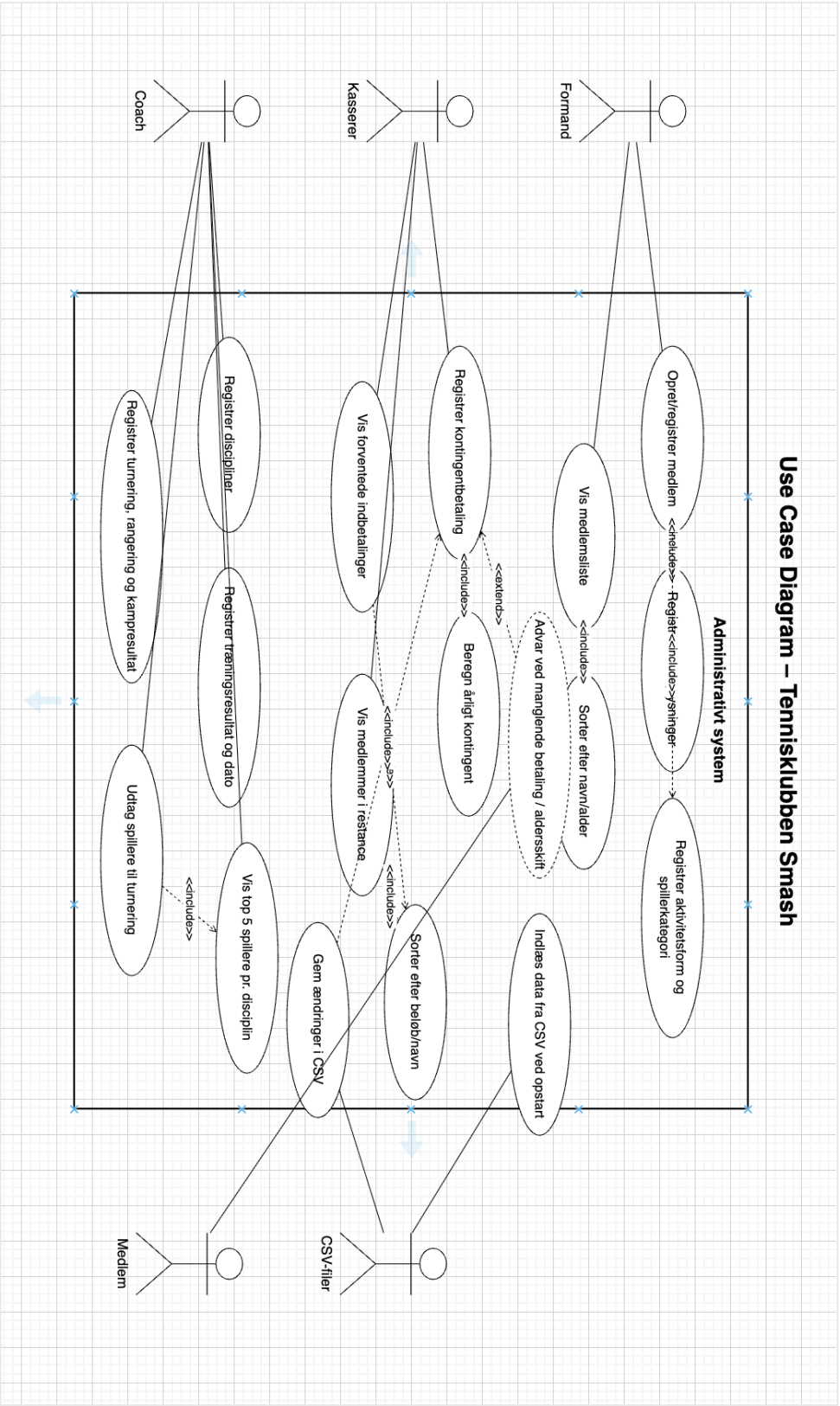
IFK-07: Filhåndtering skal anvende buffered I/O, og CPU-tidsforbruget ved læsning skal måles og sammenlignes med ikke-buffered læsning.

IFK-08: Systemet skal implementere effektiv fejlhåndtering med flere custom exceptions, inputvalidering gennem flere validatorer og error logging på flere niveauer (info, warning, error).

IFK-09: Sortering skal implementeres med både Comparable og Comparator, så objekter har en naturlig orden og samtidig kan sorteres efter alternative kriterier.

IFK-10: Centrale forretningsregler (kontingentberegning, sortering, top 5) skal dækkes af unit tests i en separat test-mappe i GitHub-repoet, og offentlige klasser og metoder skal dokumenteres med JavaDoc.

Use Case Diagram



User Stories

User story 1:

Som klubbens formand ønsker jeg at kunne registrere medlemmernes stamoplysninger, så jeg kan sortere dem efter navn og alder.

Acceptkriterier:

- Systemet gemmer: Navn, alder, memberID
- Listen over medlemmer kan sorteres alfabetisk: A-Å
- Listen over medlemmers alder kan sorteres: Høj lav / lav høj

User story 2:

Som klubbens formand ønsker jeg at kunne registrere medlemmernes ønskede aktivitetsform.

Acceptkriterier:

- Systemet skal registrere om det enkelte medlem er aktiv eller passiv
- For de aktive / passive skal der registreres om de er junior (under 18) eller senior 18+
- For de enkelte medlemmer skal der registreres om de er motionist eller konkurrencespillere
- Aktivitetsformen skal kunne ændres efter oprettelse (hvis spiller ændrer holdning/mening)
- Passive medlemmer behøver ikke yderligere kategorisering

User story 3:

Som kasserer ønsker jeg et overblik over forventede indbetalinger, samt en oversigt over medlemmer i restance.

Acceptkriterier:

- Kassereren kan se en liste over alle forventede indbetalinger
- Kassereren kan se en separat liste over medlemmer i restance
- Restance(listen) kan sorteres efter beløb

- Kasseren kan markere et medlem i restance som "Betalt" hvorefter CSV filen opdateres

User story 4:

Som coach ønsker jeg at kunne registrere spillernes træningsresultater, så jeg kan følge deres udvikling.

Acceptkriterier:

- Systemet gemmer spillernes træningsresultater og kan indlæse de eksisterende data fra CSV-filen.
- Oversigten af træningsresultat kan vælges fra start menuen.
- Træningsresultatet registreres for hver disciplin med dato.

User story 5:

Som coach ønsker jeg at kunne registrere hvilke discipliner en spiller deltager i (single, double, mixed), så spillerens aktiviteter er dokumenteret.

Acceptkriterier:

- Et medlem tildeles én disciplin: Single, Double eller Mixed Double
- Medlemmets disciplin kan ændres efter oprettelse
- Systemet skal vise hvilken disciplin en given spiller er aktiv i

User story 6:

Som coach ønsker jeg en oversigt over klubbens top 5 spillere inden for hver disciplin fordelt på junior og senior.

Acceptkriterier:

- Systemet skal vise en top-5 leaderboard: Separat fra junior og senior
- Rangeringen er baseret på bedste registrerede træningsresultater
- Hver disciplin skal vises hver for sig
- Listen skal opdateres "automatisk" når nye resultater registreres

User story 7:

Som systemadministrator ønsker jeg at systemet automatisk kan indlæse eksisterende data fra vores CSV-fil ved opstart/luk af selve systemet så der intet data går tabt mellem hver session.

Acceptkriterier:

- Vores CSV-filer skal indlæses automatisk ved opstart af systemet.
- Alle eventuelle ændringer skal gemmes automatisk når CSV-filer lukkes
- Der skal smides en Exception/fejlbesked hvis en CSV-fil mangler eller er korrumpet ved opstart
- Der skal laves en CPU-tidsmåling af det afsluttede program

User story 8:

Som system ønsker jeg at beregne kontingent ud fra medlemmets alder og medlemskab, så prisen matcher den korrekte kategori.

Acceptkriterier:

- Junior (<18) aktiv: 800 kr, Senior (18-60) aktiv: 1500 kr, Senior 60+ aktiv: 1500 kr minus 25% rabat, passiv: 250 kr
- Kassereren kan opdatere hele kontingentlisten fra menuen
- Formanden får en advarsel ved medlemmer i grænsealdrene 17 og 60, så stamoplysningerne kan opdateres inden næste kontingentkørsel

User story 9:

Som coach ønsker jeg at kunne oprette turneringer og tilmelde konkurrencespillere, så klubbens turneringsaktivitet er dokumenteret.

Acceptkriterier:

- Turneringer gemmes i Tournaments.csv med navn, dato, disciplin og deltagerliste
- Coachen kan oprette/slette turneringer og til-/afmelde medlemmer fra menuen
- Systemet indlæser eksisterende turneringer ved opstart

User story 10:

Som formand ønsker jeg at systemet giver mig en advarsel om når medlemmer skifter kontingentbetaling i forhold til deres alder.

Acceptkriterier:

- Systemet giver en advarsel når et medlem nærmer sig en aldersgrænse (17 før junior→senior, 60 før seniorrabat)
- Advarslen vises i konsollen når formanden viser medlemslisten

User story 11:

Som Formand ønsker jeg at kunne redigere og slette medlemmer fra listen (members.csv).

Acceptkriterier:

- CSV-filen, med de eksisterende medlemmers data, skal kunne indlæses.
- Ændringer i medlemmernes data gemmes automatisk til CSV-filen.
- Når et medlem bliver slettet opdateres CSV-filen automatisk.
- De medlemmer der skal slettes eller ændres bliver fundet ud fra deres medlemsID.

Trello screenshots

Sprint 1 | Planning-dokumentation

Gr/MajProjekt | Trello

Search

Create

Gr/MajProjekt | Non-US (Cont.)

- Review-dokumentation
- UI + Menustruktur
- CSV lagring og indlæsning
- Fejlhantering
- Sagning og filtrering på brugere
- Unit tests
- + Add a card

Product Backlog | User Stories

- US1: Find - Stamoplysninger
- US2: Foremand - Aktivitetsform
- US3: Kasser - Indb. & oversigt
- US4: Coach - Træ Resultater
- US5: Coach - Discipliner
- US6: Coach - Tops oversigt
- US7: Admin - CSV
- + Add a card

Sprint Backlog | Sprint 1

- Kravspec (funk. vs ikke-funk)
- Interessen-analyse
- Stakeholder Management
- Use Case Diagram
- Opsæt Træ/UML inkl. packages og projekstruktur
- + Add a card

Other/Meta/Admin/Tjeks

- Sprint Planning & review Screenstois
- Evt. andre artefakter I har brugt til projektet
- Brug OOP
- Logisk opdeling af relaterede klasser i packages
- Interface-implementering
- Brug Javadoc
- Aflæsningsformalia
- GRASP & SOLID beskrivelse
- + Add a card

Igang (On going) | Sprint 1

- Finalize user stories (incl. acceptkriterier, test/Dod)
- Domænemodel
- Klassediagram
- Sekvensdiagrammer
- Risikoanalyse og risikoplan
- + Add a card

Review (Test) | Spring 1

- + Add a card

Done | Sprint 1

- + Add a card

Inbox Planner Board Switch boards

Sprint 1 | Review-dokumentation

GrZMaProjekt

Search

Create

Sprint Backlog | Sprint 1

- + Add a card
- UserStories ...
- Stamoplysninger
- mand - sform
- sser - Indb. &
- ach - Træ.Resultater
- ach - Discipliner
- ach - Tops oversigt
- + Add a card

Igang (On going) | Sprint 1

- + Add a card
- Other/Meta/Admin/Tjeks
- Sprint Planning & review Screenshots
- 8 May
- Evt. andre artefakter I har brugt til projektet
- Brug OOP
- Interface-implementering
- Brug JavaDoc
- Aflæsningsformalia
- GRASP & SOLID beskrivelse
- + Add a card

Review (Test) | Sprint 1

- + Add a card

Done | Sprint 1

- Finalize user stories (incl. acceptkriterier, test/Dob)
- Logisk opdeling af relaterede klasser i packages
- Opsæt TrællJ inkl. packages og projekstruktur
- Stakeholder Management
- Interessent-analyse
- Risikoanalyse og riskoplan
- + Add a card

Done | Sprint 1 (cont.)

- Use Case Diagram
- Sekvensdiagrammer
- Domænemodel
- Kravsspec (funk. vs Ikke-funk)
- + Add a card

Inbox Planner Board Switch boards

Sprint 2 | Planning-dokumentation

Gr2MapProjekt

Search

Create

Team

Members

4

Share

...

ProductBacklog | Non-US (Cont.)

Review-dokumentation

Klassediagram

Søgning og filtrering på brugere

Unit tests

DOD

+ Add a card

ProductBacklog | UserStories

US4: Coach - TræResultater

US5: Coach - Discipliner

US6: Coach - Tops oversigt

US8: System - Opdatering

US9: Coach - Kampresultat

US10: "Selvvalgt udvidelse"

+ Add a card

Other/Meta/Admin/Tjeks

Sprint Planning & review Screenstris

Evt. andre artefakter I har brugt til projektet

Brug OOP

Interface-implementering

Brug JavaDoc

Aflævningsformalia

GRASP & SOLID beskrivelse

+ Add a card

Sprint Backlog | Sprint 2

Føjhåndtering

CSV lagring og indlæsning

UI + Menustruktur

+ Add a card

Ikgang (On going) | Sprint 2

US1: Fndl. - Stamoplysninger

US2: Formand - Aktivitetsform

US7: Admin - CSV

US11: Formand - slette og redigere

+ Add a card

Review (Test) | Sprint 2

+ Add a card

Done | Sprint 2

+ Add a card

Inbox

Planner

Board

Switch boards

Sprint 2 | Review-dokumentation

88

Trello

GeZMaProjekt

Try catch i filhænder ift. indlæsning af tom csv fil

1 ...

+ Add a card

ProductBacklog | Non-US (Cont.)

5 ...

Review-dokumentation

Klassediagram

CSV lagring og indlæsning

Fejlbehandling

Søgning og filtrering på brugere

+ Add a card

ProductBacklog | UserStories

6 ...

US4: Coach - Træ Resurser

US5: Coach - Disciplin

US6: Coach - Tops oversigt

US8: system - Opdatering

US9: Coach - kampresultat

US10: "Selvlagt udvalgte"

+ Add a card

Other/Meta/Admin/Tjeks

8 ...

Start Rensing & review Screenshots

0 3 days ago

Evt. andre artefakter I har brugt til projektet

Brug OOP

Interface-implementering

Brug JavaDoc

Afleveringsformåle

GRASP & SOLID beskrivelse

DD

+ Add a card

Sprint backlog | Sprint 2

0 ...

+ Add a card

Uping (On going) | Sprint 2

0 ...

+ Add a card

Review (Test) | Sprint 2

0 ...

+ Add a card

Done | Sprint 2

6 ...

US1: Ind - Startoplysninger

US2: Formand - Aktivitetsform

US3: kasser - Indb. & oversigt

US7: Admin - CSV

US11: Formand - start og nedgøre

Unit tests

+ Add a card

Done | Sprint 2 (Cont.)

2 ...

UI - Menustruktur

Fix modersnummer id det ikke giver rigtigt start eller nogle er stæket

+ Add a card

We are currently experiencing some issues. We hope to have everything back to normal shortly. <https://status.trello.com>

Search

Create

Share

Sprint 3 | Planning-dokumentation

GrZ.MajProjekt

We are currently experiencing some issues. We hope to have everything back to normal shortly. <https://status.trello.com>

Search Create

Husk/mini-todos 1 ...

- Try catch i filehandler fti. indlæsning af tom csv fil
- + Add a card

Other/Meta/Admin/Tjeks 8 ...

- Sprint Planning & review Screenshots (18 May)
- Evt. andre artefakter i har brugt til projektet
- Brug OOP
- Interface-implementering
- Brug Javadoc
- Afleveringsformalia og Presentationformalia
- GRASP & SOLID beskrivelse
- + Add a card

Sprint Backlog | Sprint 3 5 ...

- Review-dokumentation
- Klassediagram
- CSV lagning og indlæsning
- Fejlhåndtering
- Søgning og filtrering på brugere
- + Add a card

Igang (On going) | Sprint 3 3 ...

- US6: Coach - Top5 oversigt
- US8: Coach - Kampresultat
- Exception, der tjekker alder, skal tilføjes til MemberService addMember() i stedet for i FileHandlerMember.
- + Add a card

Review (Test) | Sprint 3 4 ...

- US4: Coach - Træ Resultater
- US5: Coach - Discipliner
- US8: System - Opdatering
- US10: "Selvvalgt udvidelse"
- + Add a card

Labels farver 3 ...

- Systemudvikling
- IT & Forretning
- Programmering
- + Add a card

Inbox Planner Board Switch boards

Sprint 3 | Review-dokumentation

The screenshot displays a Miro board titled "G7-MatProjekt" with a sidebar on the left containing navigation options: "Inbox", "Planer", "Board", and "Switch boards". The main workspace is divided into several columns representing different sprints and tasks:

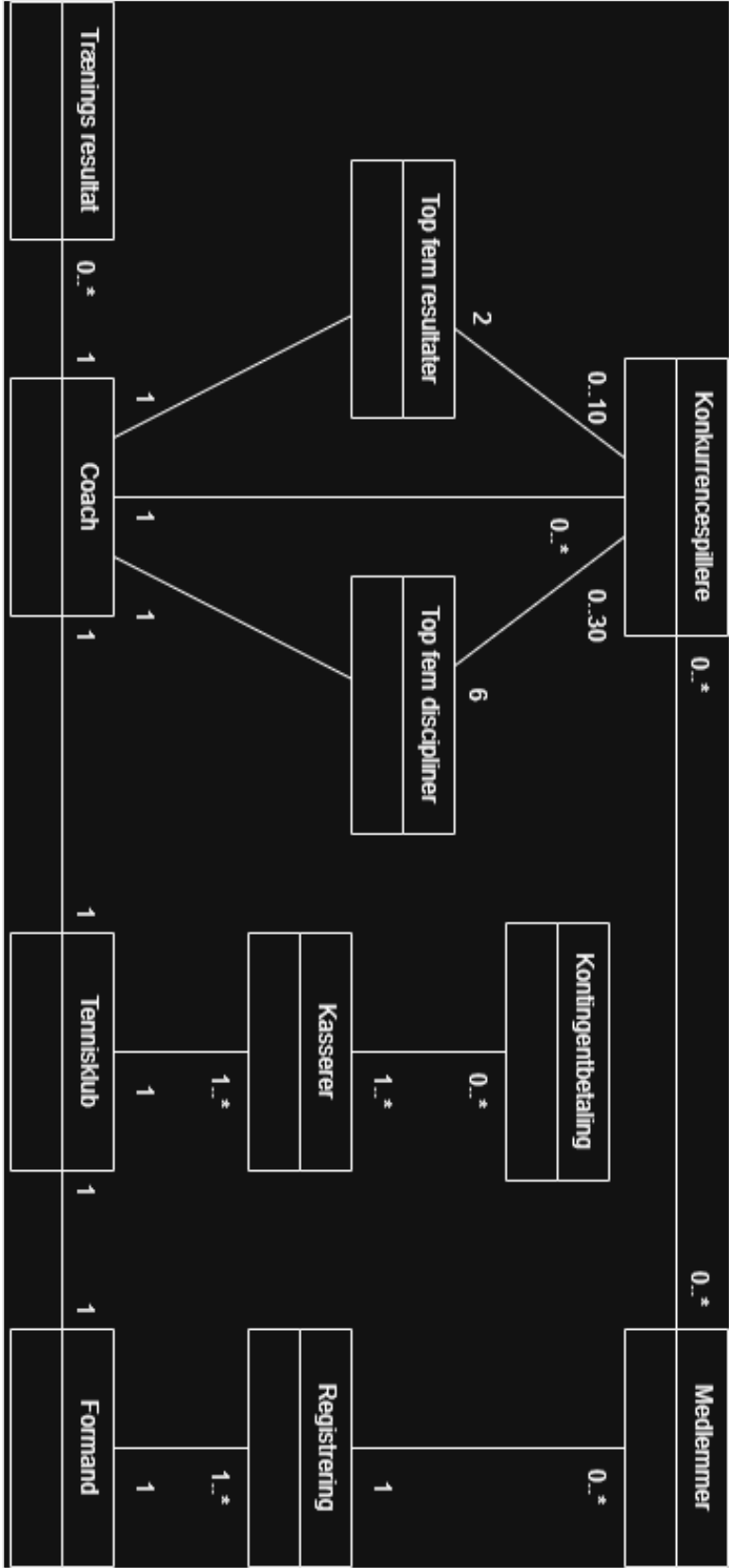
- Done | Sprint 1 (cont.)**: 4 cards including "Use Case Diagram", "Sevensdagammer", "Domænemodel", and "Kravspec (funk. vs ikke-funk)".
- Done | Sprint 1**: 6 cards including "Finalise user stories (incl. acceptance tests, testDoo)", "Logisk opdeling af reviderede klasser i packages", "Opsæt Intelij inkl. packages og projektstruktur", "Stateender Management", "Interessen-analyse", and "Risikoanalyse og risikoplan".
- Done | Sprint 2**: 6 cards including "US1: find - Stamoplysninger", "US2: Formand - Aktivitetsform", "US3: klasser - find, & oversigt", "US7: Admin - CSV", "US1: Formand - sætte og indlæse", "Unit tests", and "Unit tests".
- Done | Sprint 2 (cont.)**: 9 cards including "UI + Menustruktur", "Fik medlemsnumre så det nogle er sæt", "Interface-implementering", "Sagning og filtrering på brugere", "Exception, der tjekker alder, skal tilføjes til MemberService addMember() i stedet for i PersonerService", "CSV lagring og indlæsning", "USB Coach - kompusulat", "USB Coach - ToDS oversigt", and "Try catch i filhandler ft. indlæsning af born.csv ft".
- Done | Sprint 3**: 12 cards including "Sprint Planning & review", "EVT Kravspecifikation", "EVT FODNOTE + Review-dokumentation", "Vælder user stories inkl. lav", "Fejlhåndtering", "Opdater JAVA STRUCTURE", "Opdater sevensdagammer", "Opdater review-dokumentation", "Brug Javadoc", "EVT FODNOTE + Review-dokumentation", "Vælder user stories inkl. lav", "USB System - Opdatering", "USB Coach - 'sevensdag uvidetelse'", "Brug OOP", "GRASP & SOLID beskrivelse", and "Doo".
- Done | Sprint 3 (cont.)**: 10 cards including "Unit testing ft. at exceptions bliver castet", "Tilføj færdig til de tre UI's, samt de to metoder i ConsoleLogger klassen", "Brug Javadoc", "Klassediagram", "Opdater JAVA STRUCTURE", "Fejlhåndtering", "Opdater sevensdagammer", "Opdater review-dokumentation", "Brug Javadoc", and "EVT FODNOTE + Review-dokumentation".

At the bottom of the board, there are three diagram cards:

- Diagrammer/Bilag**: 14 cards, including "Udkast til Use Case Diagram", "Udkast til Domæne Model", and "Udkast til sevensdagammer".
- Labels/Erver**: 3 cards, including "Systemudvikling", "IT & Forretning", and "Programmering".

The bottom bar includes a "Share" button and a "Add another list" button.

Domænemodel



GRASP & SOLID

Vi kommer til at opdele GRASP og SOLID i den forstand at GRASP har 9 forskellige principper hvor hvert princip har hver deres ansvar for noget i vores kode. SOLID derimod har 5 principper som fortæller os om klasse-design, som dermed gør det nemmere at vedligeholde og potentielt udvide.

GRASP

Information Expert: Den enkelte klasse som har brugerdataen har også ansvaret for at behandle den. Her har vi i vores service package (PaymentService) som beregner medlemmernes kontingent.

Creator: Klassen som ejer objektet opretter det også. F.eks. MemberService opretter Member-objekter, fordi den skal håndtere alt hvad der angår medlemmer.

Controller: Vi har en klasse som modtager brugerens input og delegerer videre. SmashUI er rod-controlleren, der sender brugeren videre til den rolle-specifikke UI (ChairmanUI, CashierUI, CoachUI), som så kalder de relevante service-klasser. UI-klasserne indeholder ingen forretningslogik – det ligger i services.

Low Coupling: Vores klasser skal være så uafhængige af hinanden som muligt. UI-laget kalder service-klasser; service-klasserne kalder FileHandlers og model-klasser. UI-laget rører aldrig direkte ved CSV-filer eller Member-objekters interne data. Det betyder vi kan ændre hvordan en CSV bliver læst uden at røre ved UI'et.

High Cohesion: Hver klasse har hver deres formål. Et eksempel kan være vores FileHandler-klasser. De har det formål at læse og skrive til filer, hvorimod PaymentService har formålet at beregne kontingent.

SOLID

S: Single Responsibility En klasse skal have ét ansvar. Det kan f.eks. være vores FileHandler-klasser, hvor FileHandlerMembers kun håndterer medlemmer, mens FileHandlerPayment kun håndterer betalinger.

O: Open / Closed Vores Open/Closed fungerer som en klasse, hvor der er mulighed for at udvide, men der er som regel lukket for ændringer. Det kan være en af vores abstract classes som f.eks. Member. Her ville man tilføje en ny medlemstype ved at lave en subclass

og tilføje nye funktioner i den. Man laver dog ikke ændringer i vores rod, som er fra vores abstract class.

L: Liskov Substitution Liskov Substitution fungerer sådan, at vi kan bruge subklasser i stedet for parent-klassen, uden at noget går i stykker. Vores Junior og Senior arver fra Member, og overalt hvor koden forventer et Member-objekt, kan vi indsætte enten en Junior eller en Senior (fx i memberList eller når der beregnes kontingent).

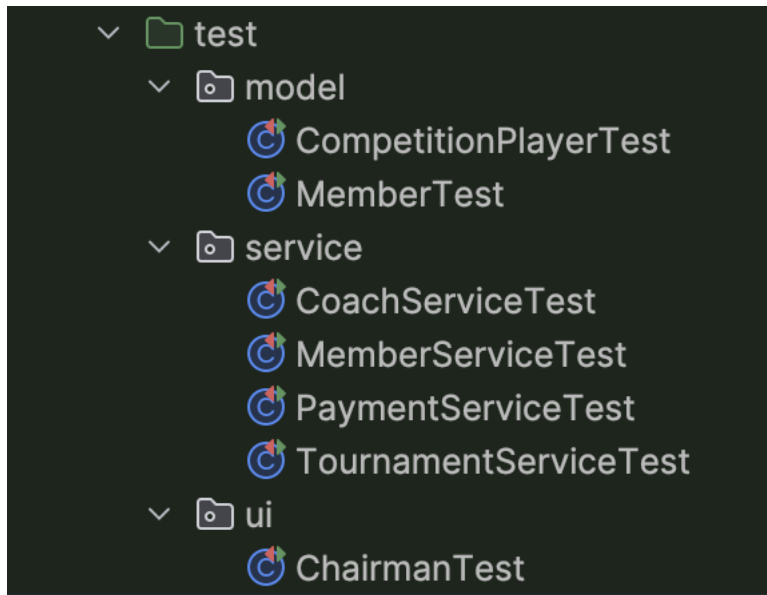
I: Interface Segregation Vi laver flere små interfaces frem for ét stort. I vores projekt har vi tre: Membership (definerer om et medlem er aktivt eller passivt), AgeValidator (validerer alder, så vi senere kan udskifte med en anden validator), og Logger (giver mulighed for forskellige log-implementeringer som ConsoleLogger). Hver klasse behøver kun det interface den faktisk bruger.

D: Dependency Inversion Klasser afhænger af interfaces frem for konkrete implementeringer. Member har et felt af typen AgeValidator (interface), ikke af typen MemberAgeValidator (konkret klasse). Det betyder vi kan udskifte validatoren uden at ændre i Member. Samme princip bruges med Logger-interfacet og ConsoleLogger.

Tests

Unit tests:

Der er implementeret unit test for en række scenarier i en separat test-mappe som kan findes i Github repo'et (jf. screenshots herunder).



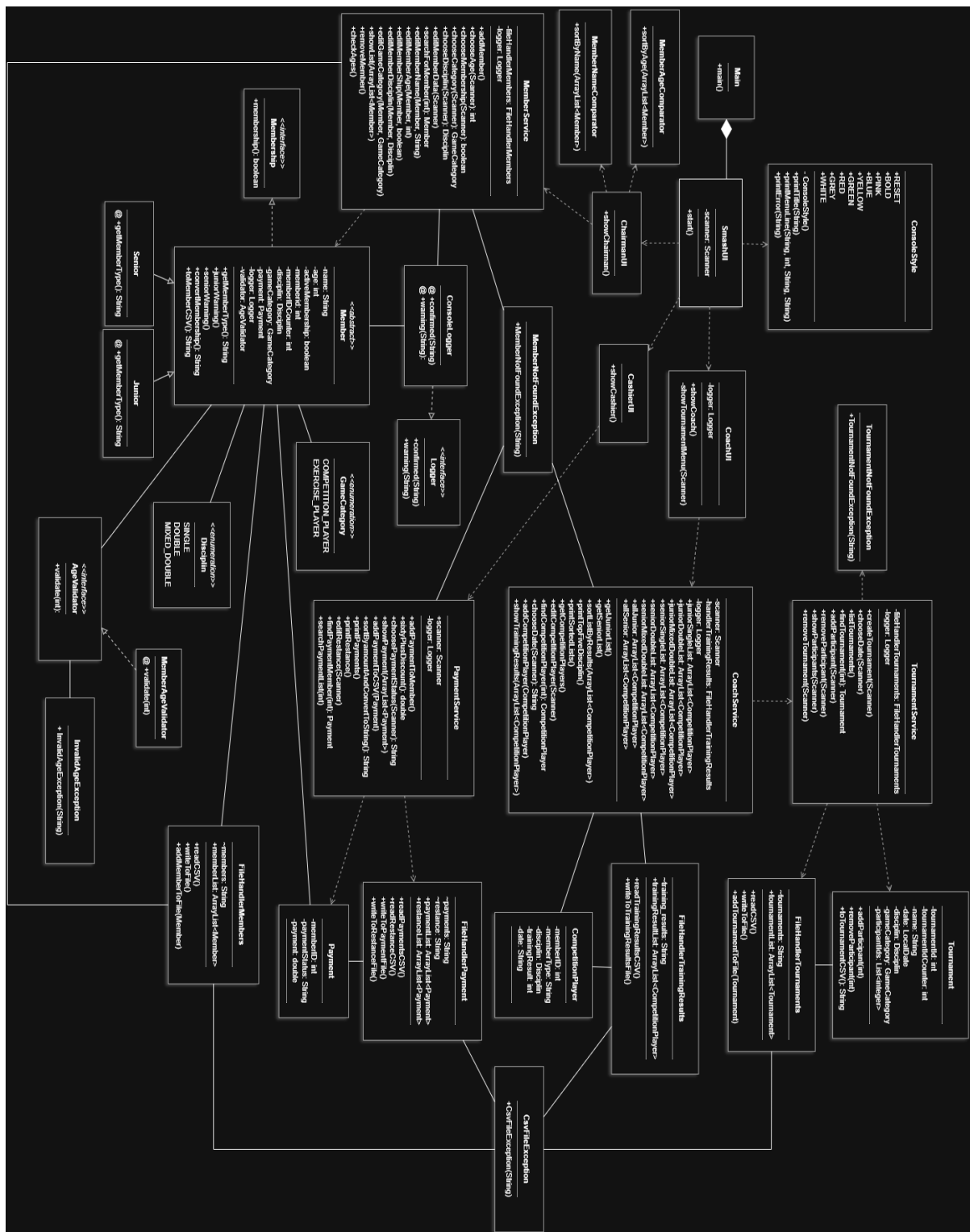
Acceptkriterer og definition of done:

Hver user story har en række acceptkriterier (jf. user story sektionen tidligere i dette dokument), hvor det for user story 1 eksempelvis er i) US1 Systemet gemmer: Navn, alder, memberID, ii) Listen over medlemmer kan sorteres alfabetisk: A-Å, og iii) Listen over medlemmers alder kan sorteres: Høj lav / lav høj

Disse acceptkriterier per user story skal ses i sammenhæng med vores generelle definition of done, som her

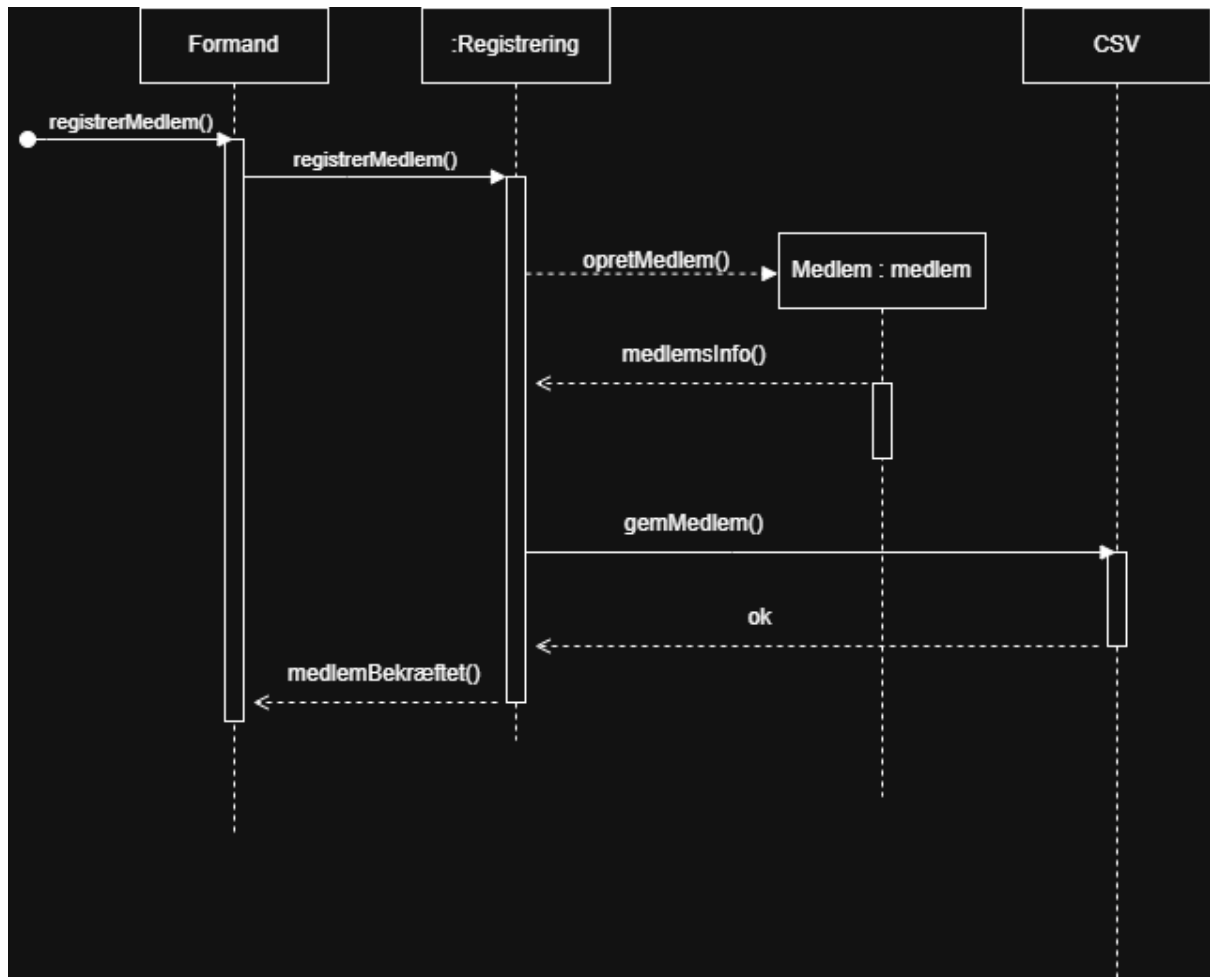
- Koden er testet af andre end dem der har kodet den
- Potentially shippable (Det er klart til at blive sendt ud til kunden (product owner))
- Pushet til GitHub
- Reviewet af andet gruppemedlem ift. kodestandard og kommentarer

Klassediagram

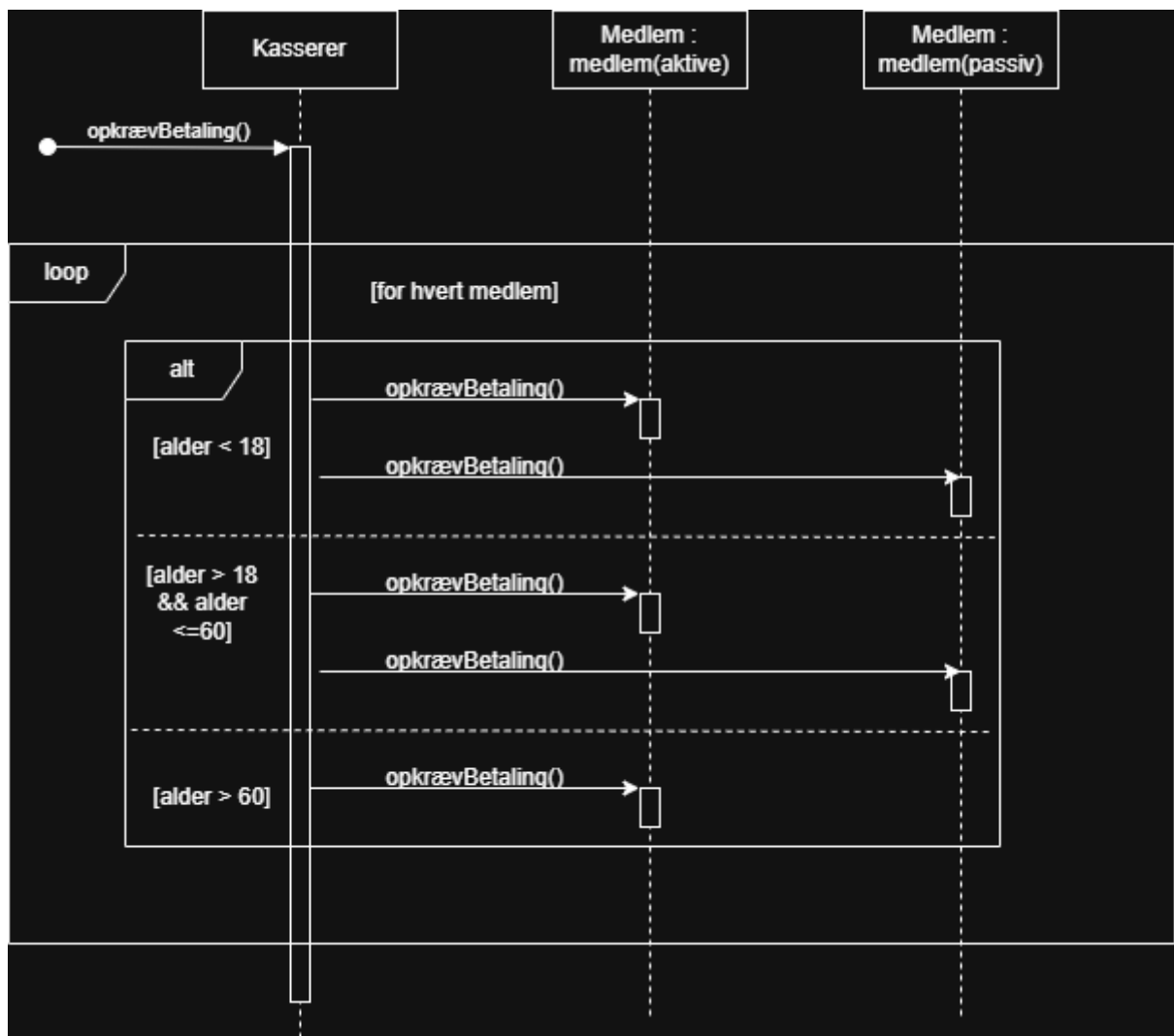


Sekvensdiagrammer

Chairman PoV



Cashier PoV



Coach PoV

